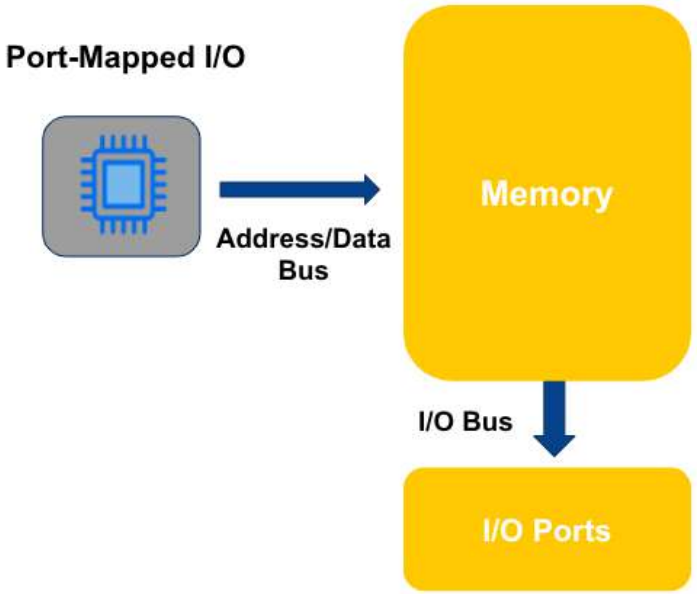
RENESAS Altium

How MCUs Access GPIO: I/O Mapping

1. Port-Mapped I/O (PMIO) / Isolated I/O:

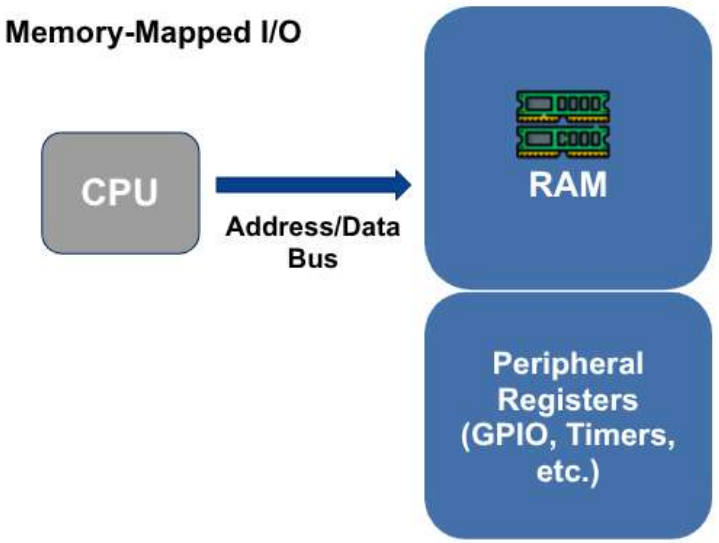
- I/O devices have a separate address space from memory.
- Requires special CPU instructions (e.g., IN, OUT on x86).
- CPU has a dedicated "I/O" pin or bus to signal an I/O operation.
- **Analogy**: A city with separate street addresses for homes and P.O. box numbers for mail.



How MCUs Access GPIO: I/O Mapping

2. Memory-Mapped I/O (MMIO):

- I/O device registers are mapped into the system's main memory address space.
- Uses standard memory access instructions (e.g., load, store).
- No special I/O instructions needed; the CPU treats the peripheral like a variable in RAM
- Dominant method in modern ARM Cortex-M MCUs like the Renesas RA6E2.
- **Analogy:** A city where every building, including the post office, shares the same street addressing system.



RENESAS

Altium

Inside the Pin: A Detailed Look

Click the next arrow to view more categories.

1

I/O Pin Pad:

The physical connection to the outside world.

2

ESD Protection Diodes

Two diodes, one connected to VCC and one to GND.

Function: To clamp dangerous electrostatic discharge voltages and protect the fragile internal logic.

3

Pull-up / Pull-down Resistors

Switch-able resistors connected to VCC (pull-up) or GND (pull-down).

Function: To provide a default logic level when the pin is configured as an input and is not being actively driven.



RENESAS

Altium

Inside the Pin: A Detailed Look

4

Schmitt Trigger Input Buffer

An input buffer with hysteresis.

Function: Cleans up noisy or slow-rising input signals and provides a stable digital logic level to the internal bus.

5

Output Driver (Push-Pull)

A pair of transistors (P-MOS connected to VCC, N-MOS connected to GND).

Function: To actively drive the output pin HIGH (sourcing current) or LOW (sinking current).

6

Control Logic

Connects to the peripheral bus and configures the state of all other components based on register values.



RENESAS

Altium

Input Mode - The "Floating" Pin Problem

Scenario: A GPIO pin is configured as an input and connected to a mechanical switch.

Two States:

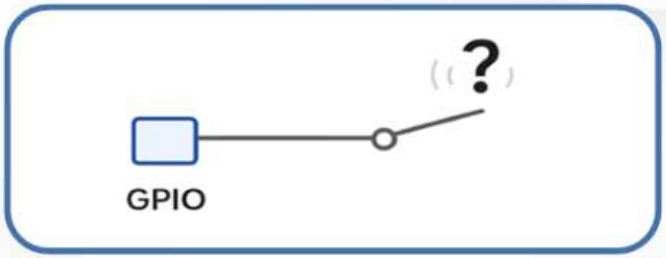
- **Switch Closed**: The pin is pulled directly to Ground. The input buffer reads a definite Logic 0 (LOW).
- **Switch Open:** The pin is connected to nothing. It's "floating".

The Problem: A floating pin acts like an antenna. It can pick up electromagnetic noise from the environment.

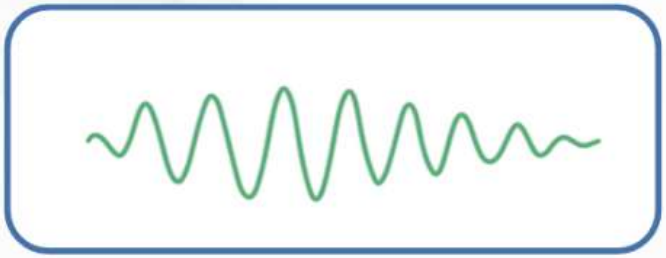
- Its voltage level is undefined and can drift randomly, causing the input buffer to rapidly and unpredictably switch between reading HIGH and LOW.

Consequence: Spurious interrupts, incorrect logic, and increased power consumption due to shoot-through current in the input buffer.

Switch Open



Floating → Unstable input



Consequences: Spurious interrupts, false readings, higher power

RENESAS

Altium

The Solution: Pull-up & Pull-down Resistors

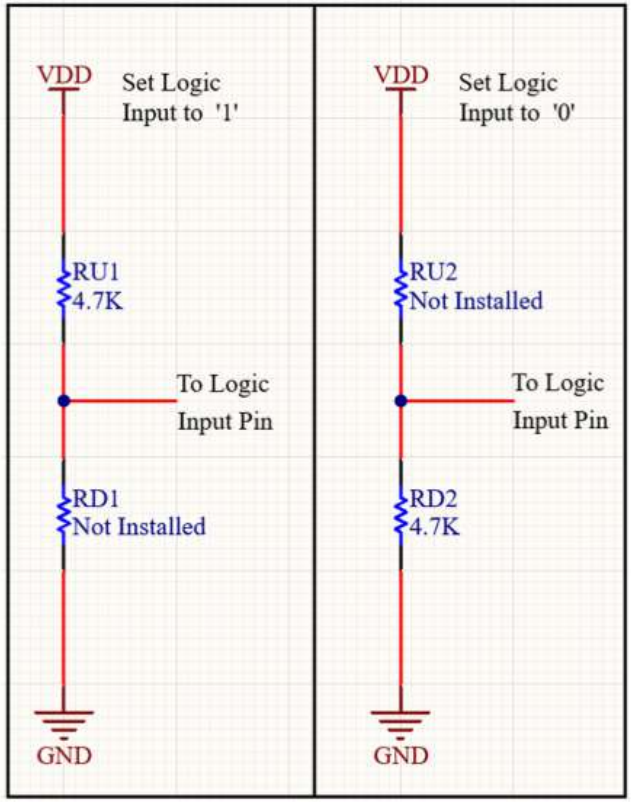
Pull-up Resistor:

- **Switch Open:** The resistor "pulls" the pin HIGH. The MCU reads a default state of Logic 1.
- **Switch Closed:** The switch provides a low-resistance path to GND, overpowering the weak pull-up. The MCU reads Logic 0. The resistor limits the current, preventing a short circuit.

Pull-down Resistor:

- **Switch Open:** The resistor "pulls" the pin LOW. The MCU reads a default state of Logic 0.
- **Switch Closed:** The switch connects the pin to VCC. The MCU reads Logic 1.

Modern MCUs (like RA6E2): Often have weak internal, software-configurable pull-up/pull-down resistors, eliminating the need for external components in many cases.

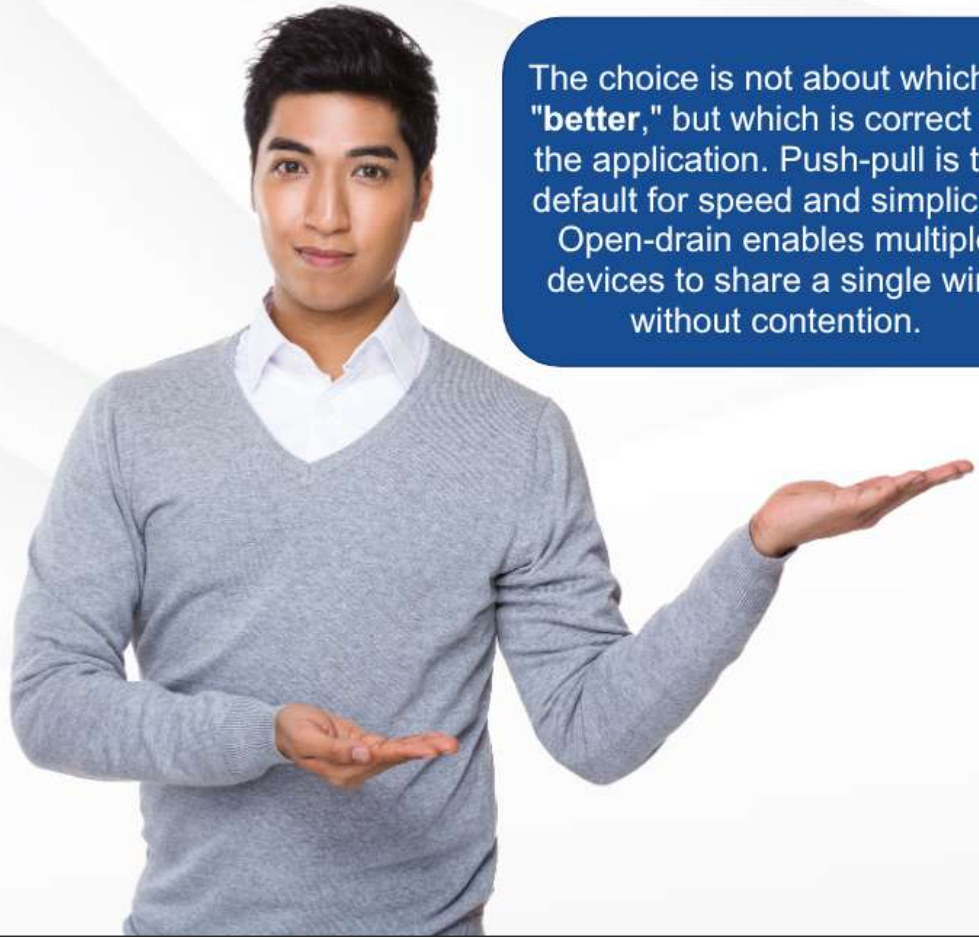


RENESAS

Altium

Output Topologies: Push-Pull vs. Open-Drain

Feature	Push-Pull Configuration	Open-Drain Configuration
Circuitry	P-MOS (to VCC) and N-MOS (to GND) transistor pair.	Single N-MOS transistor (to GND).
States	Actively drives HIGH (pushes) and LOW (pulls).	Actively drives LOW (drains). High-impedance (floating) otherwise.
External Components	None required for basic operation.	Requires an external pull-up resistor to achieve a HIGH state.
Speed	Faster signal rise times, symmetrical.	Slower rise time, determined by RC constant of pull-up and line capacitance.
Common Use Case	Driving single loads like LEDs, standard logic inputs, SPI/UART lines.	Multi-device buses (I2C), level shifting, wired-AND logic. 2



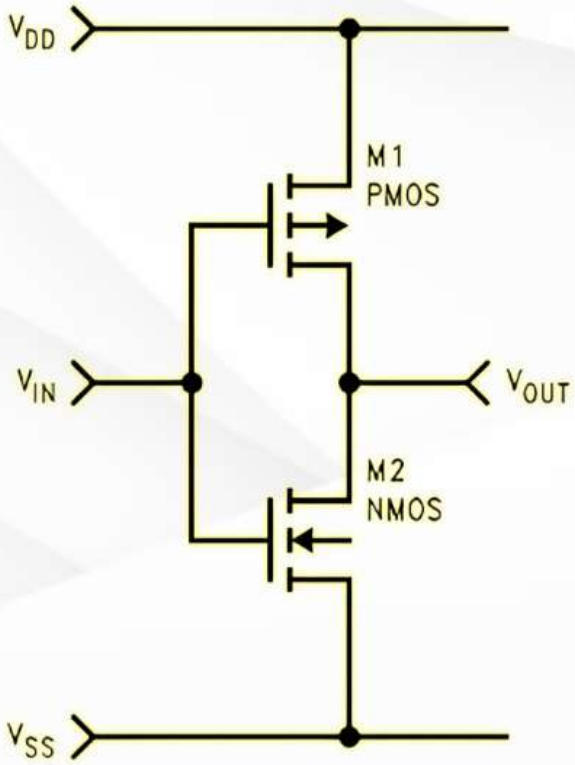
The choice is not about which is "**better**," but which is correct for the application. Push-pull is the default for speed and simplicity. Open-drain enables multiple devices to share a single wire without contention.

RENESAS

Altium

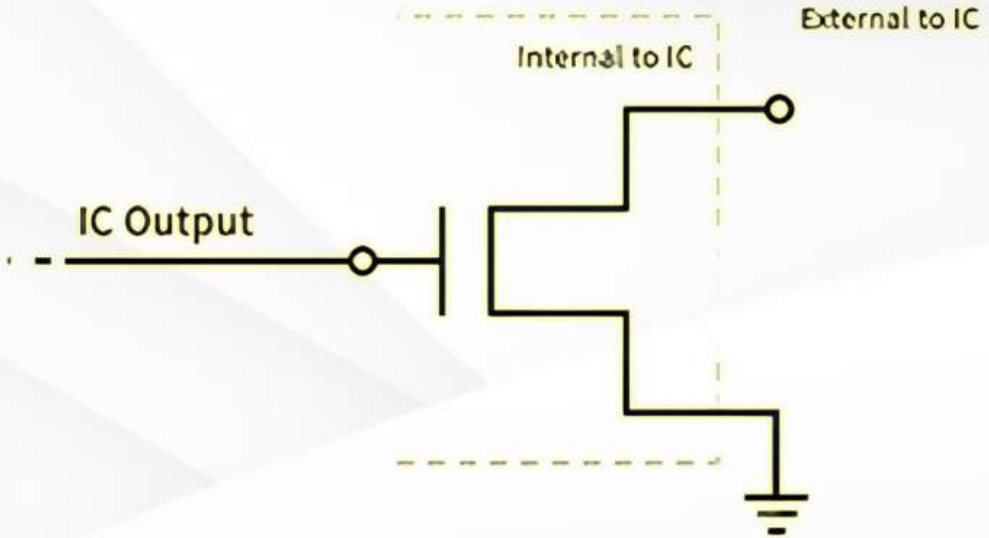
Output Topologies: Push-Pull vs. Open-Drain

Feature	Push-Pull Configuration	Open-Drain Configuration
Circuitry	P-MOS (to VCC) and N-MOS (to GND) transistor pair.	Single N-MOS transistor (to GND).
States	Actively drives HIGH (pushes) and LOW (pulls).	Actively drives LOW (drains). High-impedance (floating) otherwise.
External Components	None required for basic operation.	Requires an external pull-up resistor to achieve a HIGH state.
Speed	Faster signal rise times, symmetrical.	Slower rise time, determined by RC constant of pull-up and line capacitance.
Common Use Case	Driving single loads like LEDs, standard logic inputs, SPI/UART lines.	Multi-device buses (I2C), level shifting, wired-AND logic. 2



Output Topologies: Push-Pull vs. Open-Drain

Feature	Push-Pull Configuration	Open-Drain Configuration
Circuitry	P-MOS (to VCC) and N-MOS (to GND) transistor pair.	Single N-MOS transistor (to GND).
States	Actively drives HIGH (pushes) and LOW (pulls).	Actively drives LOW (drains). High-impedance (floating) otherwise.
External Components	None required for basic operation.	Requires an external pull-up resistor to achieve a HIGH state.
Speed	Faster signal rise times, symmetrical.	Slower rise time, determined by RC constant of pull-up and line capacitance.
Common Use Case	Driving single loads like LEDs, standard logic inputs, SPI/UART lines.	Multi-device buses (I2C), level shifting, wired-AND logic. 2



RENESAS

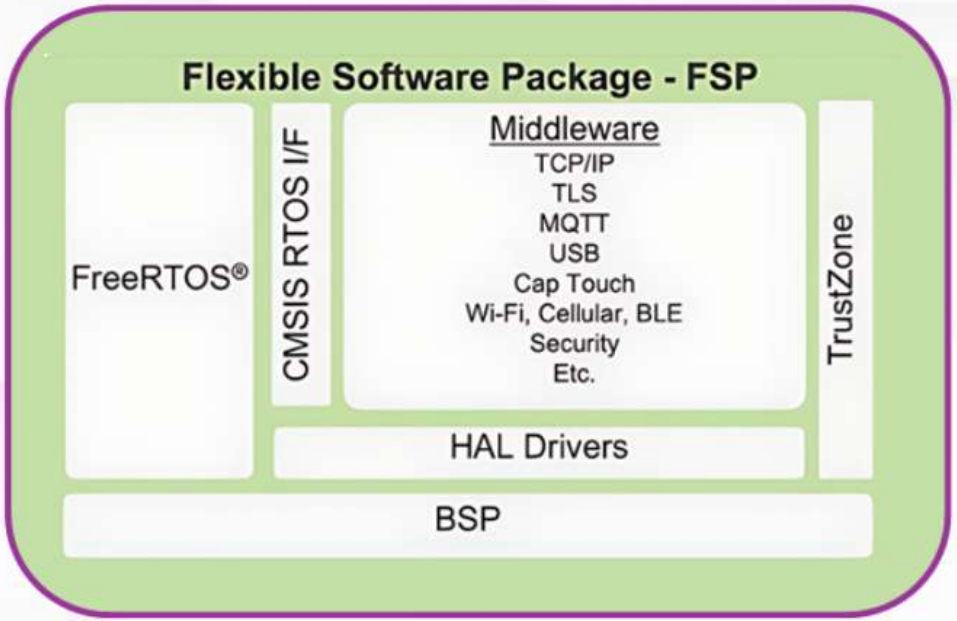
Altium

How We Program GPIO

Click the numbered button to learn more.

Level 1: Bare-Metal Register Access (The "Old" Way)

- Directly writing to memory-mapped registers.
- PORT_DIRECTION_REGISTER = 0x01; // Set pin 0 as output
- PORT_DATA_OUTPUT_REGISTER = 0x01; // Set pin 0 HIGH
- **Pros**: Maximum speed, zero overhead.
- **Cons:** Not portable, error-prone, requires deep knowledge of the hardware manual, subject to read-modify-write issues.

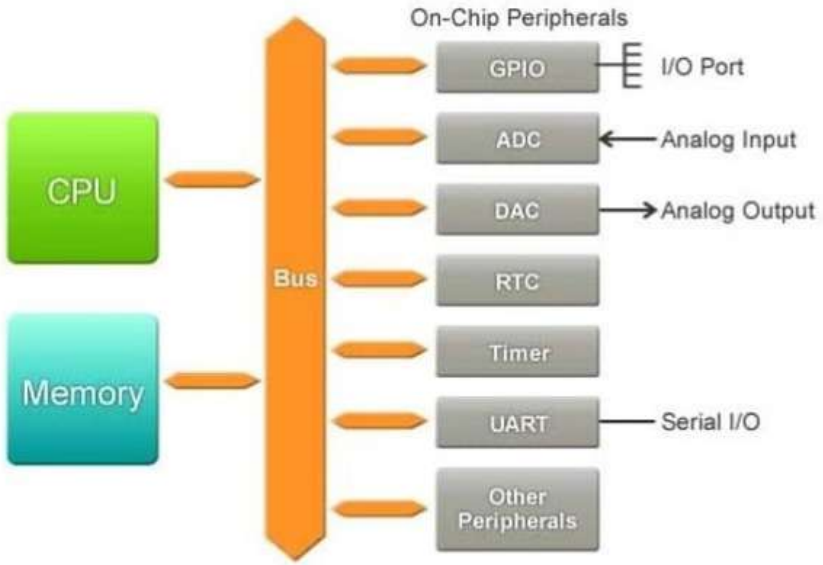


1

Renesas RA6E2 Example: FSP Pin Configuration

Tool: Renesas e² studio IDE with FSP Configurator.

1. **Open FSP Configurator:** Launch the graphical tool for the project.
2. **Select the 'Pins' Tab:** View the pinout for the R7FA6E2BB3CFM MCU.
3. **Locate Target Pin:** Find the pin connected to the on-board user LED (e.g., P400 for the blue LED on the EK-RA6E2).
4. **Configure Pin Settings:**
   - Mode: Set to Output (Initial High).
   - Symbolic Name: Give it a meaningful name, e.g., USER_LED_BLUE.
   - Drive capacity: Set to Middle.
   - Output Type: CMOS (Push-Pull).
5. **Generate Project Content**: Click the button to have FSP automatically generate the initialization code (pin_data.c).



Calculation 1: LED Current Limiting

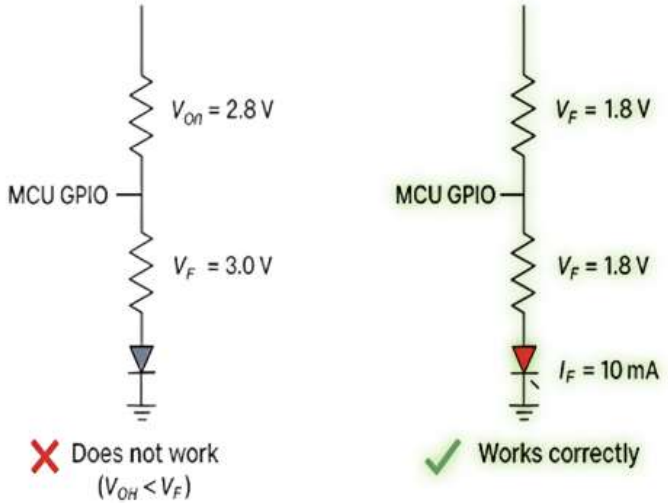
Problem: A GPIO pin can only source/sink a limited amount of current. An LED has very little internal resistance. Connecting an LED directly to a GPIO output will draw excessive current, potentially destroying the LED and/or the GPIO port.

- **Solution:** Ohm's Law (V=IR)
- **Renesas RA6E2 Example:**
  - **Goal:** Drive a standard blue LED with a forward current (IF) of 10 mA.
  - **Vsource (VOH):** The GPIO's output HIGH voltage. From the RA6E2 datasheet, VOH (min) is typically VCC−0.5V. Assuming VCC=3.3V, let's use VOH=2.8V.
  - **Vforward (VF):** The LED's forward voltage drop. For a blue LED, this is typically ~3.0V.
- **Analysis & Correction:** The required forward voltage (VF=3.0V) is greater than the GPIO's minimum source voltage (VOH=2.8V). A 3.3V GPIO cannot reliably drive a standard blue LED directly. This is a common design error. Let's recalculate for a red LED (VF≈1.8V).

Recalculation (Red LED):

$$R = \frac{(2.8 - 1.8)V}{0.01A} \qquad R = \frac{1.0V}{0.01A} = 100\Omega$$

Choose the next standard resistor value up, e.g., 120 Ω.



RENESAS Altium

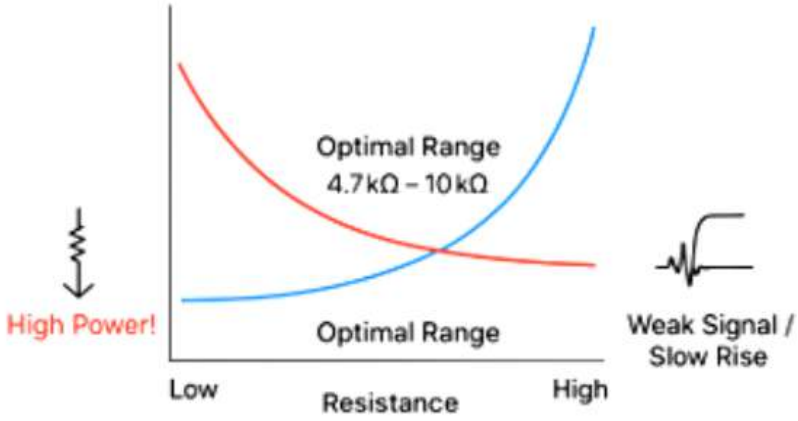
Calculation 2: Sizing Pull-up Resistors

- **The Engineering Trade-Off:**
  - **Too Low R (Strong Pull-up):** Faster signal rise time, better noise immunity. BUT, higher power consumption when the input is pulled LOW (current flows through R). P=RV2.
  - **Too High R (Weak Pull-up):** Lower power consumption. BUT, slower rise time (due to RC time constant with parasitic capacitance) and more susceptible to noise.
- **Calculation Constraints:**
  1. **Upper Bound (Max Resistance):** Must be small enough to overcome the pin's input leakage current (Ileakage) and ensure the voltage VIH(min) is met, even with noise.

$$R_{max} = \frac{(V_{CC} - V_{IH_min})}{I_{leakage}}$$

  2. **Lower Bound (Min Resistance):** Must be large enough to ensure the voltage VIL(max) is met when an external device (e.g., an open-drain sensor) pulls the line low. It must also limit current to stay within the device's sink capability (IOL_max).

$$R_{min} = \frac{(V_{CC} - V_{OL_max})}{I_{OL_max}}$$



Rule of Thumb: For most non-high-speed applications, a value between 4.7k and 10k is a safe and common choice. For high-speed buses like I2C, the value is more critical and must be calculated based on bus capacitance and speed requirements.

RENESAS Altium

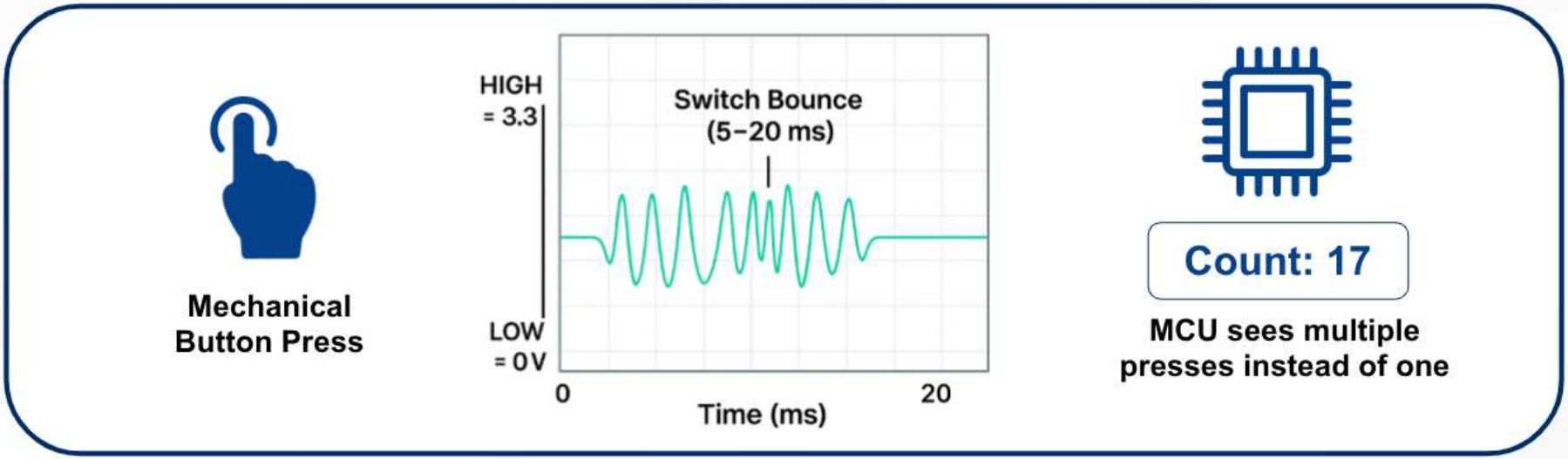
Application 1: Reading a Button & The Challenge of Switch Bounce

The Problem: Mechanical switches don't make a clean contact. The metal contacts physically bounce against each other for a few milliseconds upon closing or opening.

The MCU's Perspective: An MCU is incredibly fast. It sees this bouncing not as one press, but as dozens of rapid presses.

The Consequence: If you're trying to count presses or toggle a state, your code will behave erratically. One physical press might register as 5, 10, or even 20 events.

The Solution: We must "debounce" the switch—ignore the noisy transitions and register only one valid state change per user action.



RENESAS

Altium

Debouncing Techniques: Hardware vs. Software

Technique	Method	Pros	Cons
Hardware: RC Filter	A resistor-capacitor network smooths the signal, filtering out high-frequency bounces.	No CPU overhead; reliable and deterministic.	Requires external components; fixed debounce time.
Hardware: Schmitt Trigger	An input buffer with hysteresis provides clean logic transitions even with a slow-rising RC-filtered signal.	Excellent noise immunity; provides a very clean digital signal.	More complex/costly than a simple RC filter.
Software: Simple Delay	On first transition, wait for a fixed delay (e.g., 20ms) and then read the pin's state again.	Simple to implement; no extra hardware.	Blocking! Wastes CPU cycles; can miss other events.
Software: State Machine / Counter	Poll the input periodically. Only register a change after the input has been stable for 'N' consecutive samples.	Non-blocking; flexible debounce time; no extra hardware.	More complex to code; consumes some CPU cycles for polling.
Hybrid: MCU Peripheral (ICU)	Use a built-in peripheral like the Renesas ICU with its digital noise filter.	Hardware-accelerated; low CPU overhead; configurable.	Uses up a dedicated peripheral; not available on all pins.

Application 2: PWM for "Analog" Output

Faking Analog: Pulse Width Modulation (PWM)

Concept: GPIOs are digital—they can only be fully ON or fully OFF. But by switching a pin ON and OFF very rapidly, we can control the average voltage, effectively creating an analog-like output.

Key Terms:

- **Period**: The total time for one ON/OFF cycle.
- **Duty Cycle**: The percentage of the period that the signal is HIGH.

Application: Controlling LED brightness, motor speed, or generating simple audio tones.

Renesas RA6E2:

- While you can implement PWM in software by toggling a GPIO in a timed loop ("bit-banging"), it's extremely inefficient and timing is imprecise.
- The correct way is to use a dedicated hardware peripheral like the General PWM Timer (GPT), which can generate precise PWM signals on a GPIO pin with zero CPU intervention after configuration.



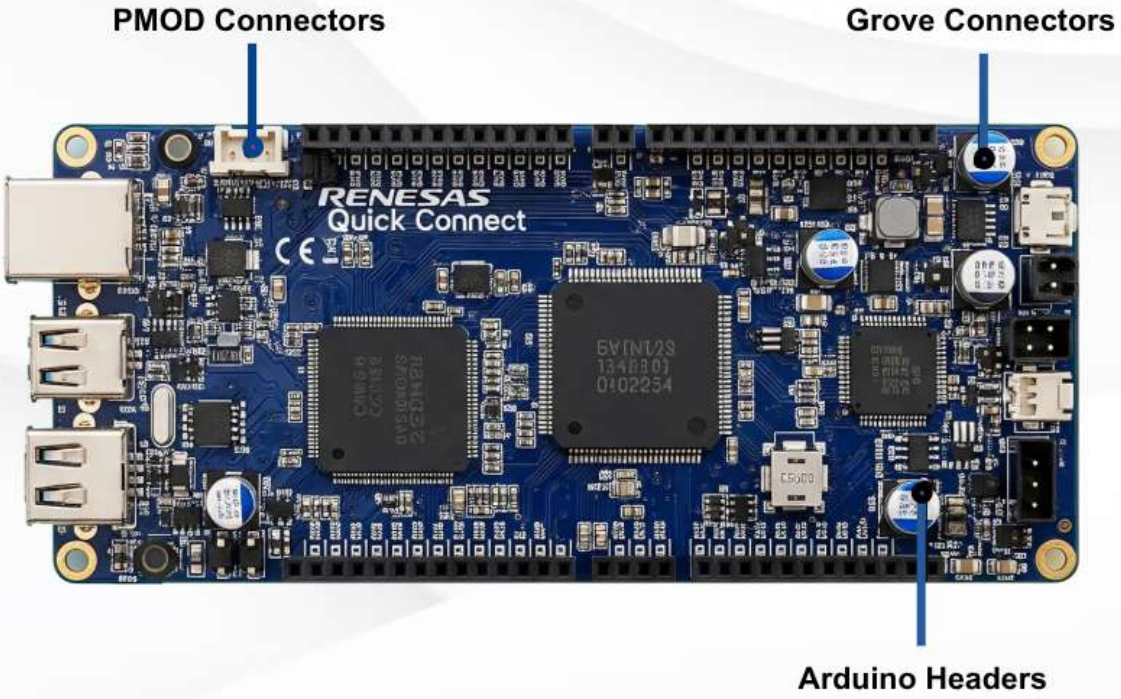
From Physical Pins to Intelligent Conversations

Recap of Module 3: Physical Interfaces

1. **PMOD:** An open standard from Digilent using 6, 8, or 12-pin connectors.
2. **Grove:** A 4-pin plug-and-play system from Seeed Studio.
3. **Arduino Headers:** A ubiquitous standard for add-on "shields".

Real-World Example: Renesas Quick Connect Kit

- Integrates PMOD, Grove, and Arduino connectors directly on the board.
- This approach accelerates development by allowing rapid connection to a vast ecosystem of peripherals.



The Core Question: We know how to connect... but how do they talk?